



SMART IRRIGATION AND WATER MANAGEMENT SYSTEM

EXERCISE NO – 07

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

7. SMART IRRIGATION AND WATER MANAGEMENT SYSTEM

1. Project overview:

This project implements a Smart Irrigation and Water Management System that automates the watering of an agricultural plot based on real-time environmental conditions. The system continuously monitors soil moisture, ambient temperature, humidity, and rainfall, and switches an irrigation pump on or off automatically to keep soil moisture within an optimal, configurable range — without requiring manual intervention.

It is designed as a low-cost, field-deployable module of a larger precision-agriculture and farm water-management platform, where zone-level sensor data can be used to reduce water wastage, prevent over- or under-irrigation, and give farmers or facility managers remote visibility into field conditions.

The prototype was built and debugged end-to-end on a NodeMCU (ESP8266) development board with a capacitive soil moisture sensor, a DHT11 temperature/humidity sensor, a rain sensor module, a relay-driven water pump, and a status LED, with cloud connectivity added via the Thing Speak IoT platform for remote data logging and visualization.

2. Problem Statement:

Agriculture accounts for roughly 70% of global freshwater withdrawal, and conventional irrigation practices — fixed watering schedules or manual judgement — routinely over- or under-water crops. This leads to:

- Significant water wastage from irrigating fields that are already sufficiently moist
- Crop stress and yield loss from delayed watering during genuinely dry conditions
- Wasted irrigation cycles that run during or immediately after rainfall
- No remote visibility into field/soil conditions for farmers managing multiple plots

There is a need for a low-cost, easily deployable IoT sensing-and-actuation unit that can continuously monitor soil and environmental conditions, automatically control irrigation based on real, measured need, and optionally upload data to the cloud for remote monitoring and trend analysis.

3. Proposed Solution:

A single-zone smart irrigation node built around a NodeMCU (ESP8266) microcontroller was proposed and implemented with the following capabilities:

- Continuous soil moisture sensing using a capacitive soil moisture probe, sampled every 2 seconds
- Ambient temperature and humidity sensing using a DHT11 sensor, to support future evapotranspiration-based scheduling
- Rainfall detection using a rain sensor module, to automatically suppress irrigation during or after rain
- Closed-loop, threshold-based pump control: a relay-driven water pump switches ON when soil moisture drops below a dry threshold and OFF once it recovers above a wet threshold
- A software safety cutoff that limits maximum continuous pump runtime per irrigation cycle, to protect against sensor faults or stuck logic
- Serial diagnostics for real-time debugging and verification during development
- Cloud data logging to Thing Speak over Wi-Fi, enabling remote visualization of soil moisture, temperature, humidity, and pump-activity trends.

4. System Architecture:

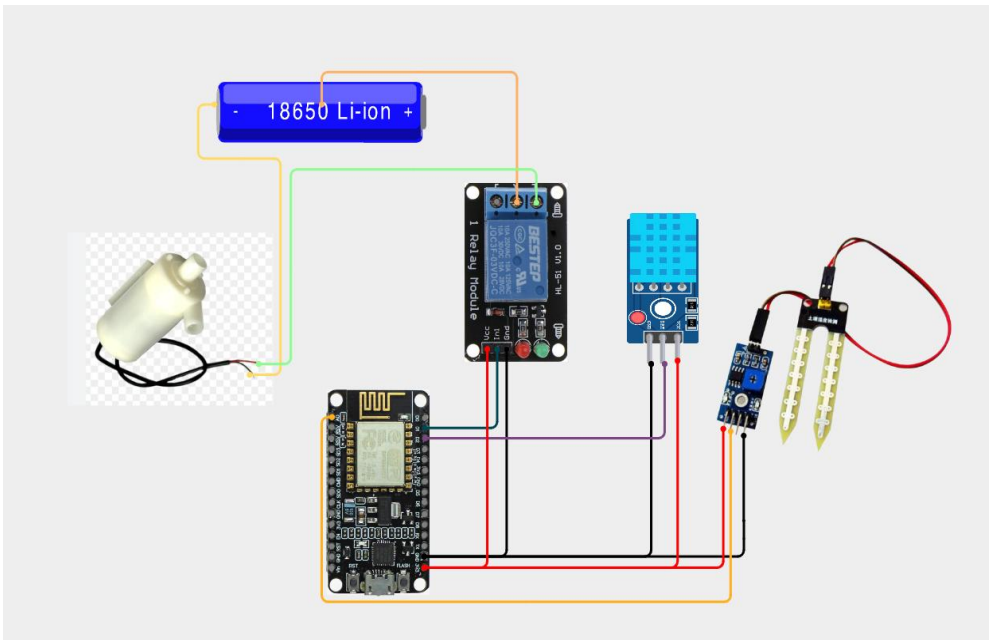
The system follows a layered IoT architecture:

- **Sensing Layer:** Soil moisture probe, DHT11, and rain sensor capture field and ambient conditions.
- **Edge/Control Layer:** NodeMCU reads all sensors, evaluates irrigation threshold logic, and drives the relay/pump and LED directly — this works even without network connectivity.
- **Connectivity Layer:** ESP8266 Wi-Fi radio connects to a local network/hotspot (2.4 GHz only).
- **Actuation Layer:** A relay module switches the submersible water pump; a status LED mirrors pump state for local visual confirmation.
- **Cloud Layer:** Sensor and pump-status readings are pushed to a Thing Speak channel via HTTP GET requests to the Thing Speak Update API.
- **Visualization Layer:** Thing Speak's built-in channel charts display live and historical soil moisture, temperature, humidity, and pump-activity trends.

5.Circuit Table:

Component	Pin	ESP 32
Soil Moisture Sensor	VCC	3.3V
Soil Moisture Sensor	DATA	GPIO 34
Soil Moisture Sensor	GND	GND
Relay Module	IN	GPIO 26
Relay Module	VCC	5V
Relay Module	GND	GND
Water Pump	+/-	Relay Output

6. Circuit Design:



7. Algorithm:

Main control loop logic (executes continuously once the board boots):

1. Initialize serial output, relay pin, LED pin, rain-sensor pin, and DHT sensor.
2. Every 2 seconds, read soil moisture (analog), temperature/humidity (DHT11), and rain status (digital).
3. If the DHT read fails (NaN), log an error but continue evaluating irrigation logic using the last valid soil/rain readings.
4. If the pump is OFF and soil moisture < SOIL_DRY_PCT and it is not currently raining, turn the pump ON and record the start time.
5. If the pump is ON and (soil moisture $\geq$ SOIL_WET_PCT OR rain is detected), turn the pump OFF immediately.
6. If the pump has been ON continuously for longer than PUMP_MAX_RUN_MS, force it OFF as a safety cutoff, regardless of the soil reading.
7. Mirror pump state on the status LED (ON when pumping, OFF otherwise).
8. (Cloud-enabled variant) Every 20 seconds, if Wi-Fi is connected, upload the latest soil moisture, temperature, humidity, and pump state to the Thing Speak channel via an HTTP GET request; otherwise skip and retry on the next interval.
9. Repeat indefinitely.

8. Coding:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <DHT.h>
#define WIFI_SSID "K-APIL"
#define WIFI_PASS "kapil.2333"
#define IO_USERNAME "Kapilparamasivam"
#define IO_KEY "aio_LUjQ50sb43WYuxYc1oLL7ESLA03X"
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
AdafruitIO_Feed *soilFeed = io.feed("soil-moisture");
AdafruitIO_Feed *tempFeed = io.feed("temperature");
AdafruitIO_Feed *humidityFeed = io.feed("humidity");
AdafruitIO_Feed *pumpFeed = io.feed("pump-status");
#define SOIL_PIN A0
#define DHT_PIN 4
#define RELAY_PIN
#define DHTTYPE DHT11
DHT dht(DHT_PIN, DHTTYPE);
```

```

int MOISTURE_LIMIT = 30;
#define RELAY_ON LOW

#define RELAY_OFF HIGH
unsigned long lastTime = 0;
void setup()
{
  Serial.begin(115200);
  pinMode(SOIL_PIN, INPUT);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, RELAY_OFF);
  dht.begin();
  Serial.println();
  Serial.println("SMART IRRIGATION SYSTEM");

  io.connect();
  while (io.status() < AIO_CONNECTED)
  {
    Serial.print(".");
    delay(500);
  }
  Serial.println();
  Serial.println("Adafruit IO Connected!");
}
void loop()
{
  io.run();
  if (millis() - lastTime >= 5000)
  {
    lastTime = millis();
    int rawValue = analogRead(SOIL_PIN);
    int moisture = map(rawValue, 1023, 0, 0, 100);
    moisture = constrain(moisture, 0, 100);
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();
    if (isnan(temperature) || isnan(humidity))
    {
      Serial.println("DHT11 ERROR");
      return;
    }
    String pump;
    if (moisture < MOISTURE_LIMIT)
    {
      digitalWrite(RELAY_PIN, RELAY_ON);
      pump = "ON";
    }
  }
}

```

```

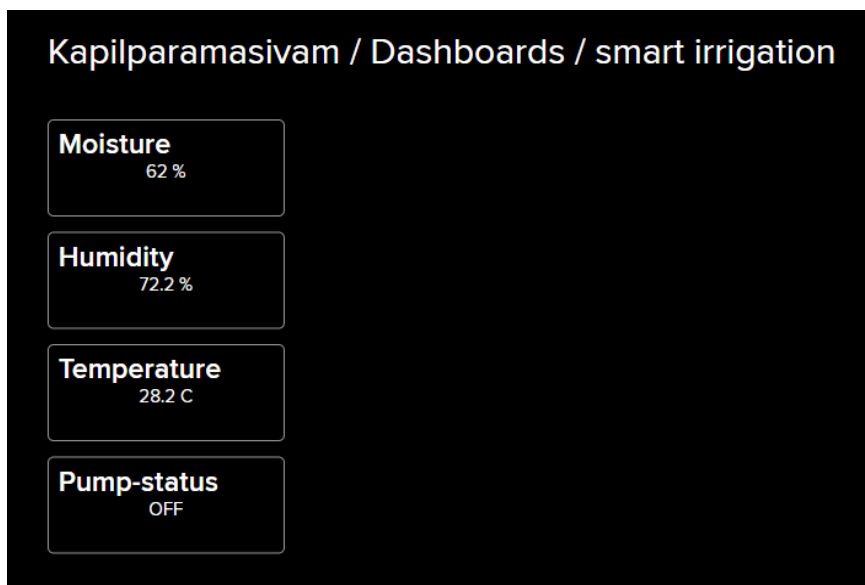
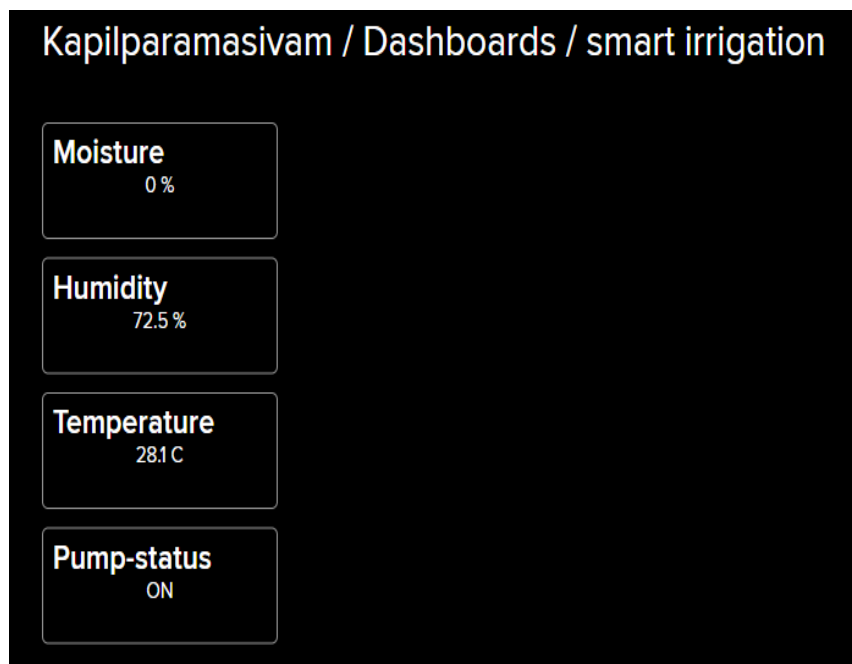
    }
    else
    {
        digitalWrite(RELAY_PIN, RELAY_OFF);
        pump = "OFF";
    }
    String soilValue = String(moisture) + " %";
    String tempValue = String(temperature, 1) + " C";
    String humidityValue = String(humidity, 1) + " %";
    Serial.println();
    Serial.println();
    Serial.print("Soil Raw    = ");
    Serial.println(rawValue);
    Serial.print("Soil Moisture = ");
    Serial.println(soilValue);
    Serial.print("Temperature  = ");
    Serial.println(tempValue);
    Serial.print("Humidity     = ");
    Serial.println(humidityValue);
    Serial.print("Pump Status  = ");
    Serial.println(pump);
    Serial.println();
    Serial.println("Sending...");
    soilFeed->save(soilValue);
    delay(300);
    tempFeed->save(tempValue);
    delay(300);
    humidityFeed->save(humidityValue);
    delay(300);
    pumpFeed->save(pump);
    Serial.println("Sent!");
}
}

```

9. System Working:

On power-up, the NodeMCU initializes all peripherals and confirms the relay defaults to OFF. It then enters a continuous sensing-and-control loop:

- Every 2 seconds, the soil moisture probe, DHT11, and rain sensor are polled.
- Valid readings are printed to the Serial Monitor for live diagnostics (e.g., "Soil: 28.4% Temp: 27.9C Humidity: 64.2% Rain: NO").
- If soil moisture drops below the configured dry threshold (35%) and no rain is detected, the relay energises the pump and the status LED turns on.
- The pump automatically switches off once soil moisture recovers above the wet threshold (60%), or immediately if rain is detected, or after the maximum safety runtime is reached, whichever comes first.
- In the cloud-enabled configuration, the board also connects to a Wi-Fi access point (2.4 GHz only) and pushes the latest readings and pump state to ThingSpeak every 20 seconds, where they appear as live charts on the channel dashboard.




```
=====
Soil Raw      = 387
Soil Moisture = 62 %
Temperature   = 28.2 C
Humidity      = 72.2 %
Pump Status   = OFF
=====
```

```
Sending...
Sent!
```

```
=====
Soil Raw      = 391
Soil Moisture = 62 %
Temperature   = 28.2 C
Humidity      = 72.2 %
Pump Status   = OFF
=====
```

```
Sending...
Sent!
```

```
=====
Soil Raw      = 399
Soil Moisture = 61 %
Temperature   = 28.2 C
Humidity      = 72.2 %
Pump Status   = OFF
=====
```

```
Sending...
Sent!
```

```
=====
Soil Raw      = 1024
Soil Moisture = 0 %
Temperature   = 28.2 C
Humidity      = 72.2 %
Pump Status   = ON
=====
```

```
Sending...
Sent!
```

```
=====
Soil Raw      = 1024
Soil Moisture = 0 %
Temperature   = 28.2 C
Humidity      = 72.1 %
Pump Status   = ON
=====
```

```
Sending...
Sent!
```

10. Bill of Materials:

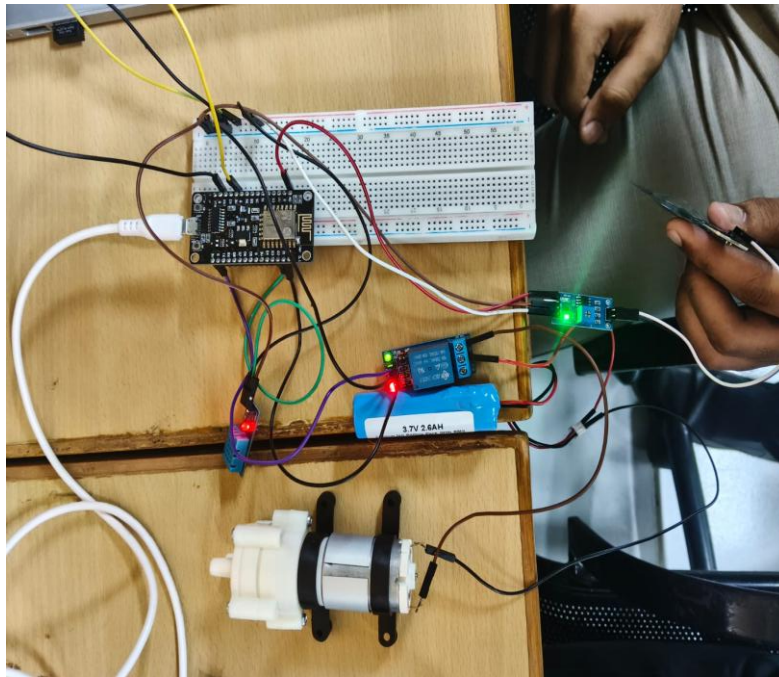
COMPONENT	QUANTITY
ESP8266 NodeMCU	1
Soil Moisture Sensor	1
Relay Module	1
Mini Water Pump	1
DHT 11 Sensor	1
Breadboard	1
Jumper Wires	Required

11. Prototype Implementation:

The prototype was assembled and debugged iteratively, with the following notable steps and issues resolved during implementation:

- Soil moisture calibration: raw ADC readings were captured in fully dry air (~850) and fully submerged in water (~350), and these were used as the map() endpoints for the 0–100% moisture scale, since capacitive sensor output varies between units and soil types.
- An initial version used a resistive soil moisture probe; it was replaced with a capacitive sensor after observing electrode corrosion and drifting readings over repeated wet/dry cycles.
- The relay module's active-low signal logic was initially misread as active-high, causing the pump to run continuously at startup; this was corrected by explicitly setting RELAY_OFF (HIGH) during setup() before entering the main loop.
- Board profile was set to "Generic ESP8266 Module," which required using raw GPIO numbers (GPIO2, GPIO14, GPIO13, GPIO12) in code instead of Arduino D-pin aliases, since those aliases are only defined under NodeMCU-specific board profiles.
- The pump was initially planned to be powered directly from the NodeMCU's 3V3 rail; this was corrected to a separate 5V supply switched through the relay's NO contact, after recognising the pump's stall current would exceed the onboard regulator's capacity.
- A post-boot brownout, similar to that seen in earlier ESP8266 sensing work, was anticipated and mitigated with the same standard hardware fix: a bulk capacitor across 3V3/GND.
- The rain sensor module's sensitivity threshold (onboard potentiometer) required manual tuning so that light condensation did not falsely suppress irrigation.

- A software safety cutoff (PUMP_MAX_RUN_MS) was added after a test run in which a stuck-wet false soil reading kept the pump inactive for an extended period; the cutoff instead guards the opposite failure mode — a stuck-dry reading — from running the pump indefinitely.



12.Results & Performance Analysis:

Once wiring, calibration, and relay-logic issues were resolved, the system performed as designed:

- The soil moisture sensor produced consistent readings every 2 seconds, visible on the Serial Monitor, and correctly tracked drying trends over successive test cycles.
- The pump reliably switched ON when soil moisture fell below 35% and switched OFF once it recovered above 60%, with the status LED accurately mirroring pump state.
- Rain-sensor integration was verified by manually wetting the sensor board during an active dry-soil condition, which correctly halted the pump immediately, confirming the rain-override logic worked as intended.
- Wi-Fi connectivity was successfully established, confirmed via a clean "CONNECTED!" status with an assigned IP address.
- Cloud upload to Thing Speak was implemented and configured to respect the platform's 15-second minimum update interval, using a 20-second upload cycle; readings for soil moisture, temperature, humidity, and pump status were confirmed appearing on the channel dashboard.
- The safety cutoff was validated by artificially holding the soil reading below threshold for longer than PUMP_MAX_RUN_MS, confirming the pump forced itself off as expected.

Overall, the prototype demonstrates a working local sensing-and-actuation pipeline with a clear path to full cloud-connected operation.

Remaining work includes multi-zone soil-probe calibration, integrating weather-forecast data to further reduce unnecessary irrigation cycles, and extending the system to a closed-loop, multi-zone water-management platform with per-zone scheduling and reporting.